

CHAPTER 14

Principles of Learning

Machine learning is, for the most part, sub-divided into three components based on the approach to the learning problem. The three predominant categories of learning are the supervised, unsupervised, and reinforcement learning schemes. In this chapter, we will go over supervised learning schemes in detail and also touch upon unsupervised and reinforcement learning schemes to a lesser extent.

The focus on supervised learning is for a variety of reasons. Firstly, they are the predominant techniques used for building machine learning products in industry; secondly, as you will soon learn, they are easy to ground truth and assess their performances before being deployed as part of a large-scale production pipeline. Let's examine each of the three schemes.

Supervised Learning

To easily understand the concept of supervised learning, let's revisit the problem of identifying spam emails from a set of emails. We will use this example to understand key concepts that are central to the definition and the framing of a supervised learning problem, and they are

- Features
- Samples
- Targets

For this contrived example, let's assume that we have a dictionary of the top 4 words in the set of emails and we record the frequency of occurrence for each email sample. This information is represented in a tabular format, where each feature is a column and the rows are email samples. This tabular representation is called a dataset. Figure 14-1 illustrates this depiction.

	Features				Targets
	\$\$\$	article	ML	BENEFICIARY	Target
email sample 1	5	1	0	6	spam
email sample 2	0	4	7	0	regular
email sample 3	0	3	5	0	regular
...	4	0	0	4	spam
email sample n	8	1	0	5	spam

Figure 14-1. Dataset representation

The fundamental concept behind supervised machine learning is that each sample is associated with a target variable, and the goal is to teach the computer to learn the patterns from the dataset features that results in a target as a prediction outcome. The columns of a dataset in machine learning are referred to as features; other names you may find commonly used are **variables** or **attributes** of the dataset, but in this book, we will use the term features to describe the measurement units of a data sample. Moreover, the samples of a dataset are also referred to as rows, data points, or observations, but we will use the term samples throughout this book.

Hence, in supervised learning, a set of features are used to build a learning model that will predict the outcome of a target variable as shown in Figure 14-1.

Next, we will cover important modeling considerations for building supervised learning models.

Regression vs. Classification

In supervised learning, we typically have two types of modeling task, and they are **regression** and **classification**.

Regression

The supervised learning problem is a regression task when the values of the target variable are real-valued numbers.

Let's take, for example, that we are given a housing dataset and are asked to build a model that can predict the price of a house. The dataset, for example, has features such as the price of the house, the number of bedrooms, the number of bathrooms, and the total square feet. Let's illustrate how this dataset will look like with a contrived example in Figure 14-2.

Features			Target feature
#bedrooms	#bathrooms	sq. ft	price
4	6	3	18.3
2	4	1	15.2
...	...	...	...
5	8	5	24.7

Figure 14-2. Regression problem: housing dataset

From the learning problem, the features of the dataset are the number of bedrooms, the number of bathrooms, and the square foot of the floor area, while the target feature is the price of the house. The use case presented in Figure 14-3 is framed as a **regression task** because the target feature is a **real-valued number**.

Classification

In a classification task, the target feature is a label denoting some sort of class membership. These labels are also called categorical variables, because they represent labels that belong to two or more categories. Also, no natural ordering exists between the categories or labels.

As an example, suppose we are given a dataset containing the heart disease diagnosis of patients, and we are asked to build a model to predict if a patient has a heart disease or not. Like the previous example, let's assume the dataset has features blood pressure, cholesterol level, heart rate, and heart disease diagnosis. A contrived illustration of this example is shown in Figure 14-3.

Features			Target feature
blood pressure	cholesterol level	heart rate	heart disease
4	6	3	0
2	4	1	1
...	...	...	...
5	8	5	0

In Target feature,
0: no heart disease
1: heart disease

Figure 14-3. Classification task: heart disease dataset

From the table in Figure 14-3, the **target variable denotes a class membership of heart disease or no heart disease**; hence, the target is categorical and can be termed as a **classification problem**.

How Do We Know that Learning Has Occurred?

This question is vital to determine if the learning algorithm can learn a useful pattern between the input features and the targets. Let's create a scenario that will give us better insights into appraising the question of determining when learning has occurred.

Assume a teacher takes a physics class for 3 months, and at the end of each session, the teacher administers a test to ascertain if the student has learned anything.

Let's consider two different scenarios the teacher might use in evaluating the students:

1. The teacher evaluates the student with the exact word-for-word questions that were used as sample problems while teaching.
2. The teacher evaluates the student with an entirely different but similar set of sample problems that are based on the principles taught in class.

In which of these subplots can the teacher ascertain that the student has learned? To figure this out, we must consider the two norms of learning:

1. **Memorization:** In the first subplot, it will be incorrect for the teacher to form a basis for learning because the student has seen and most likely memorized the examples during the class sessions. Memorization is when the exact snapshot of a sample is stored for future recollection. Therefore, it is inaccurate to use samples used in training to carry out learning evaluation. In machine learning, this is known as *data snooping*.
2. **Generalization:** In the second subplot, the teacher can be confident that the assessment serves as an accurate test to evaluate if the student has learned from the session. The ability to use the principles learned to solve previously unseen samples is known as *generalization*.

Hence, we can conclude that learning is the ability to generalize to previously unseen samples.

Training, Test, and Validation Datasets

The goal of supervised machine learning is to be able to predict or classify the targets on unseen examples correctly. We can misjudge the performance of our learning models if we evaluate the model performance with the same samples used to train the model as explained previously.

To properly evaluate the performance of a learning algorithm, we need to set aside some data for testing purposes. This held-out data is called a **test set**.

Another situation arises when we have trained the model on a dataset, and we now need to improve the performance of the model by adjusting some of the learning algorithm's parameters.

We cannot use the test set for model tuning because if we do that, the model's parameters are trained with the test dataset rendering it unusable as an unseen held-out sample for model evaluation. Hence, it is typical to divide the entire dataset into

- The training set (to train the model)
- The validation set (to tune the model)
- The test set (to evaluate the effectiveness of the model)

A common and straightforward strategy is to split 60% of the dataset for training, 20% for validation, and the final 20% for testing. This strategy is popularly known as the 60/20/20 rule. We will discuss more sophisticated methods for resampling (i.e., using subsets of available data) for machine learning later in this chapter. See Figure [14-4](#).

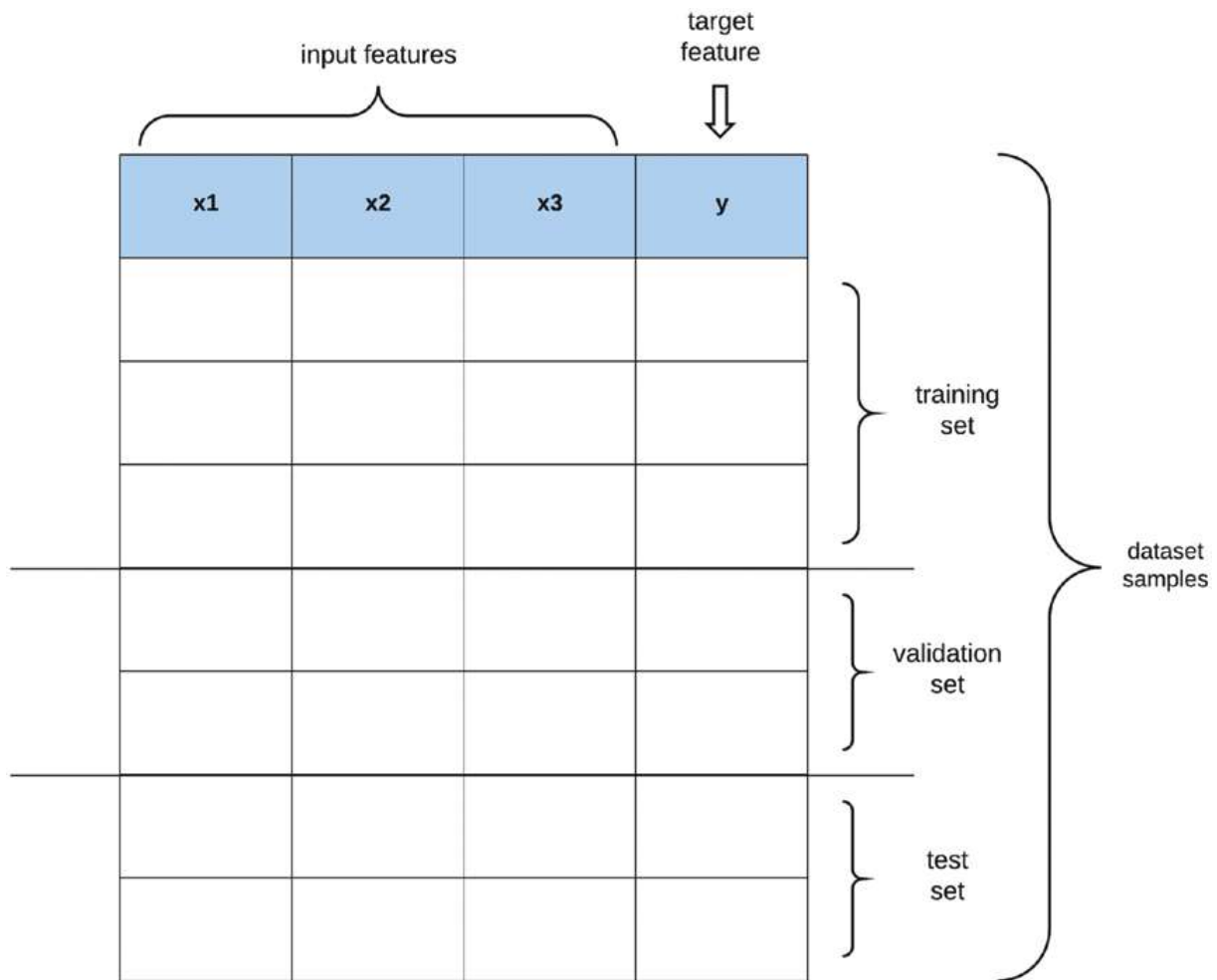


Figure 14-4. Training, test, and validation set

Bias vs. Variance Trade-Off

The concept of bias vs. variance is central to machine learning and is critical to understanding how the model is performing, as well as in suggesting the direction in which to improve the model.

A model is said to have high **bias** when it oversimplifies the learning problem or when the model fails to accurately capture the complex relationships that exist between the input features of the dataset. High bias makes the model unable to generalize to new examples.

High variance, on the other hand, is when the model learns too closely the intricate patterns of the dataset input features, and in the process, it *learns the irreducible noise of the dataset samples*. When the learning algorithm learns very closely the patterns of the training samples, including the noise, it will fail to generalize when exposed to previously unseen data.

Hence, we observe that there is a need to strike the right balance between bias and variance, and often it is down to the skill of the model builder to discover this middle ground. However, there exists practical rules of thumb for finding the right trade-off between bias and variance.

How Do We Recognize the Presence of Bias or Variance in the Results?

High bias is observed when the model performs poorly on the trained data. Of course, it will also perform poorly (or even worse) on the test data with high prediction errors. When high bias occurs, it can be said that the model has underfit the data. High variance is observed when the trained model learns the training data very well but performs poorly on unseen (test) data. In the event of high variance, we can say that the model has overfit the dataset.

The graph in Figure 14-5 illustrates the effect of bias and variance on the quality/performance of a machine learning model. In Figure 14-6, the reader will observe that there is a sweet spot somewhere in the middle where the model has good performances on both the training and the test datasets.

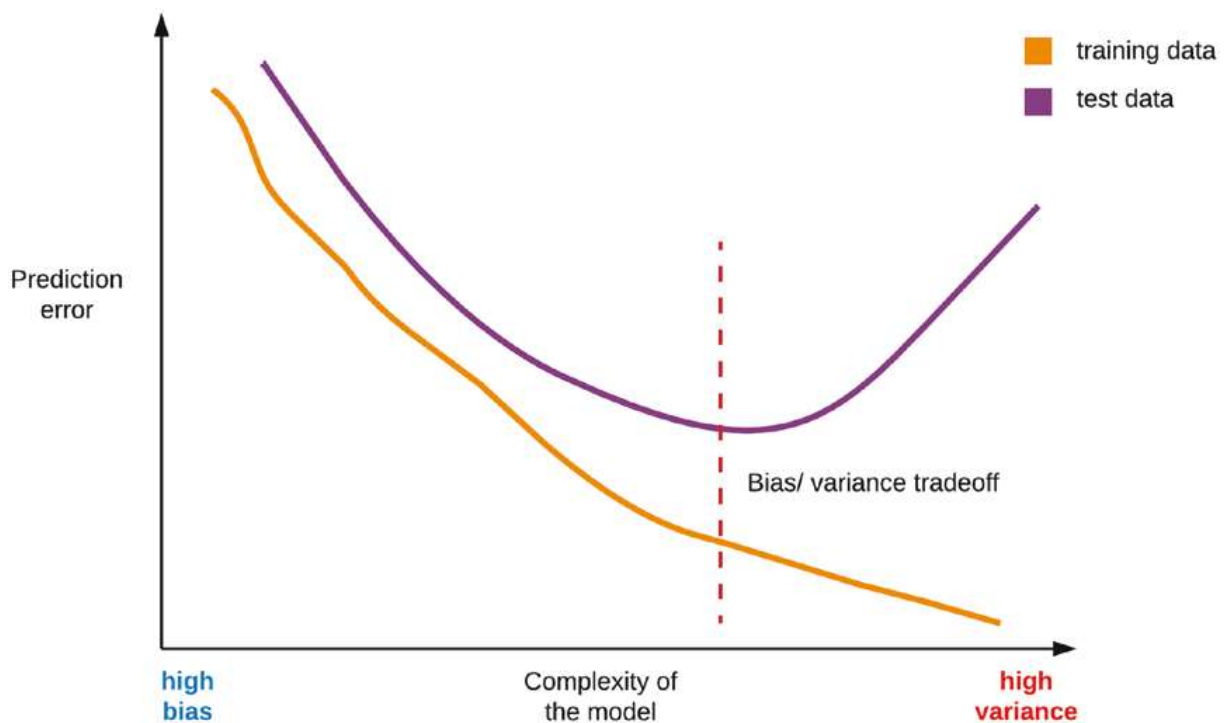


Figure 14-5. *Bias and variance*

To recap, our goal is to have a model that strikes a balance between high bias and high variance. Figure 14-6 provides further illustration on the effects of models with high bias and variance on a dataset. As seen in the image to the left of Figure 14-6, we want to have a model that can generalize to previously unseen example, such a model should have good prediction accuracy.

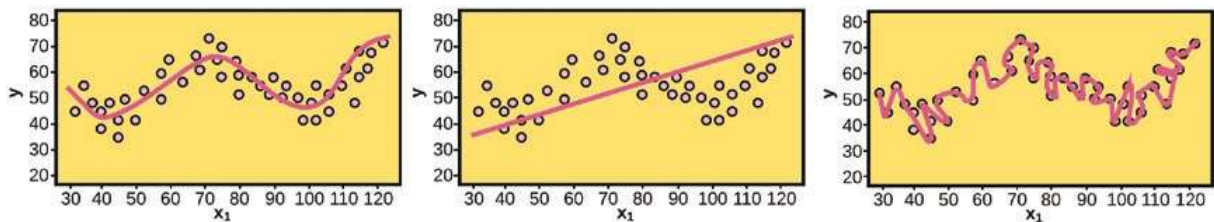


Figure 14-6. *Left: Good fit. Center: Underfit (high bias). Right: Overfit (high variance)*

Evaluating Model Quality

Evaluation metrics give us a way to quantitatively evaluate how well our model is performing. The model's performance on the training data is evaluated to get the training set accuracy, while its performance on the test data is evaluated to get the test data accuracy when the model predicts the targets of previously unseen examples. Evaluation on test data helps us to know the true performance measure of our model.

The learning problem determines the type of evaluation metric to use. As an example, for regression prediction problems, it is common to use the root mean squared error (RMSE) to evaluate the magnitude of the error made by the model. For classification problems, one of the common evaluation metrics is to use a confusion matrix to get a picture of how many samples are correctly classified or misclassified. From the confusion matrix, it is possible to derive other useful metrics for evaluating classification problems such as accuracy, precision, and recall.

The following are the evaluation metrics for machine learning that we will consider in this text:

Classification

- Confusion matrix
- Area under ROC curve (AUC-ROC)

Regression

- Root mean squared error (RMSE)
- R-squared (R^2)

Let's go through each.

Classification Evaluation Metrics

In this section, we'll briefly explain performance metrics for classification machine learning tasks.

Confusion Matrix

The confusion matrix is a popular evaluation metric for gleaning insights into the performance of a classification supervised machine learning model. It is represented as a table with grid-like cells. In the case of a two-class classification problem, the columns

of the grid are the actual positive and negative class values of the target feature, while the rows are the predicted positive and negative class values of the targets. This is illustrated in Figure 14-7.

		Actual Value	
		Positive	Negative
Predicted Value	Positive	TP	FP
	Negative	FN	TN

Figure 14-7. *Confusion matrix*

There are four primary values that can be gotten directly from examining the confusion matrix, and they are the **true positive**, the **false positive**, the **true negative**, and the **false negative values**. Let's examine each of them briefly:

- True positive: True positive is the number of samples predicted to be positive (or true) when the actual class is positive.
- False positive: False positive is the number of samples predicted as positive (or true) when the actual class is negative.
- True negative: True negative is the number of samples predicted to be negative (or false) and the actual class is negative.
- False negative: False negative is the number of samples predicted to be negative (or false) when the actual class is positive.

From the four primary values, we have three other measures that provide more information on the performance of our model. These are accuracy, the positive predictive value (or precision), and sensitivity (or recall). Let's explain them briefly:

- Accuracy: Accuracy is the fraction of correct predictions made by the learning algorithm. It is represented as the ratio of the sum of true positive, *TP*, and true negative, *TN*, to the total population.

$$accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

- Precision or positive predictive value: Precision is the ratio of true positive, TP , to the sum of true positive, TP , and false positive, FP . In other words, precision measures the fraction of results that are correctly predicted as positive over all the results that the algorithm predicts as positive. The sum $TP + FP$ is also called the predicted positive condition.

$$precision = \frac{TP}{TP + FP}$$

- Recall or sensitivity: Recall is the ratio of true positive, TP , to the sum of true positive, TP , and false negative, FN . In other words, recall retrieves the fraction of results that are correctly predicted as positive over all the results that are positive. The sum $TP + FN$ is also known as condition positive.

$$recall = \frac{TP}{TP + FN}$$

To put this concept together, let's revisit the example heart disease dataset. Suppose we are to predict if a patient will be diagnosed with a heart disease or not, assume we have 50 samples in the dataset, of which 20 are diagnosed with heart disease and the remaining 30 are not. Of the 30 samples that do not have a disease diagnosis, the learning algorithm rightly identifies 25, while of the 20 samples that have a disease diagnosis, the learning algorithm correctly identifies 15.

Let's represent this information in a confusion matrix (see Figure 14-8) and calculate the necessary statistical measures to evaluate the algorithm performance.

		Actual Value	
		Positive [have disease]	Negative [no disease]
Predicted Value	Positive [have disease]	15	5
	Negative [no disease]	5	25

Figure 14-8. Confusion matrix example

From the data in Figure 14-8, we can calculate the **accuracy**, **precision**, and **recall**.

- Accuracy =

$$\frac{15 + 25}{15 + 5 + 5 + 25} = \frac{40}{50} = \frac{4}{5}$$

- Precision =

$$\frac{15}{15 + 5} = \frac{3}{4}$$

- Recall =

$$\frac{15}{15 + 5} = \frac{3}{4}$$

Hence, our algorithm is 80% accurate, with a precision of 75% and a recall of 75%.

Area Under the Receiver Operating Curve (AUC-ROC)

The area under the receiver operating characteristic (ROC) curve, also known as the AUC-ROC for short, is another widely used metric for evaluating classification machine learning problems. A significant feature of the AUC-ROC metric is that it can be a good metric for evaluating datasets with imbalanced classes.

An imbalanced class is when one of the outcome targets has far more samples than another target. A typical example of this can be seen in a fraud identification dataset where the samples of no fraud will be vastly more than the samples with fraud.

To better understand AUC-ROC, let us derive two (2) relevant formulas from our confusion matrix, and they are the **True negative rate (TNR)** (also known as **specificity**) and the **False positive rate** (also known as **fall-out**).

Specificity is the fraction of results that are **correctly predicted as negative** over all the results that are negative, whereas fall-out is the fraction of results that are **wrongly predicted as positive** over all the results that are negative. Fall-out is also represented as $(1 - \text{specificity})$.

This is further illustrated in Figure 14-9.

		Actual Value	
		Positive [have disease]	Negative [no disease]
Predicted Value	Positive [have disease]	TP	FP
	Negative [no disease]	FN	TN

specificity = $\frac{\text{TN}}{\text{FP} + \text{TN}}$	fall-out = $\frac{\text{FP}}{\text{FP} + \text{TN}}$
---	--

Figure 14-9. *Specificity and fall-out*

The ROC Space

The ROC or receiver operating characteristic space is a 2-D graph that **plots** the cumulative probability distribution of the sensitivity (i.e., the probability distribution of making the correct prediction) on the y axis and the cumulative probability distribution of the fall-out (i.e., the probability distribution of a false alarm) on the x axis.

A few notable details about the ROC space is that

- The area of the square space is 1 because the x and y axes range from 0 to 1, respectively.
- The diagonal line drawn from point $(x = 0, y = 0)$ to $(x = 1, y = 1)$ represented pure chance or a random guess. It is also known as the line of no discrimination.

These expressions are further illustrated in Figure 14-10.

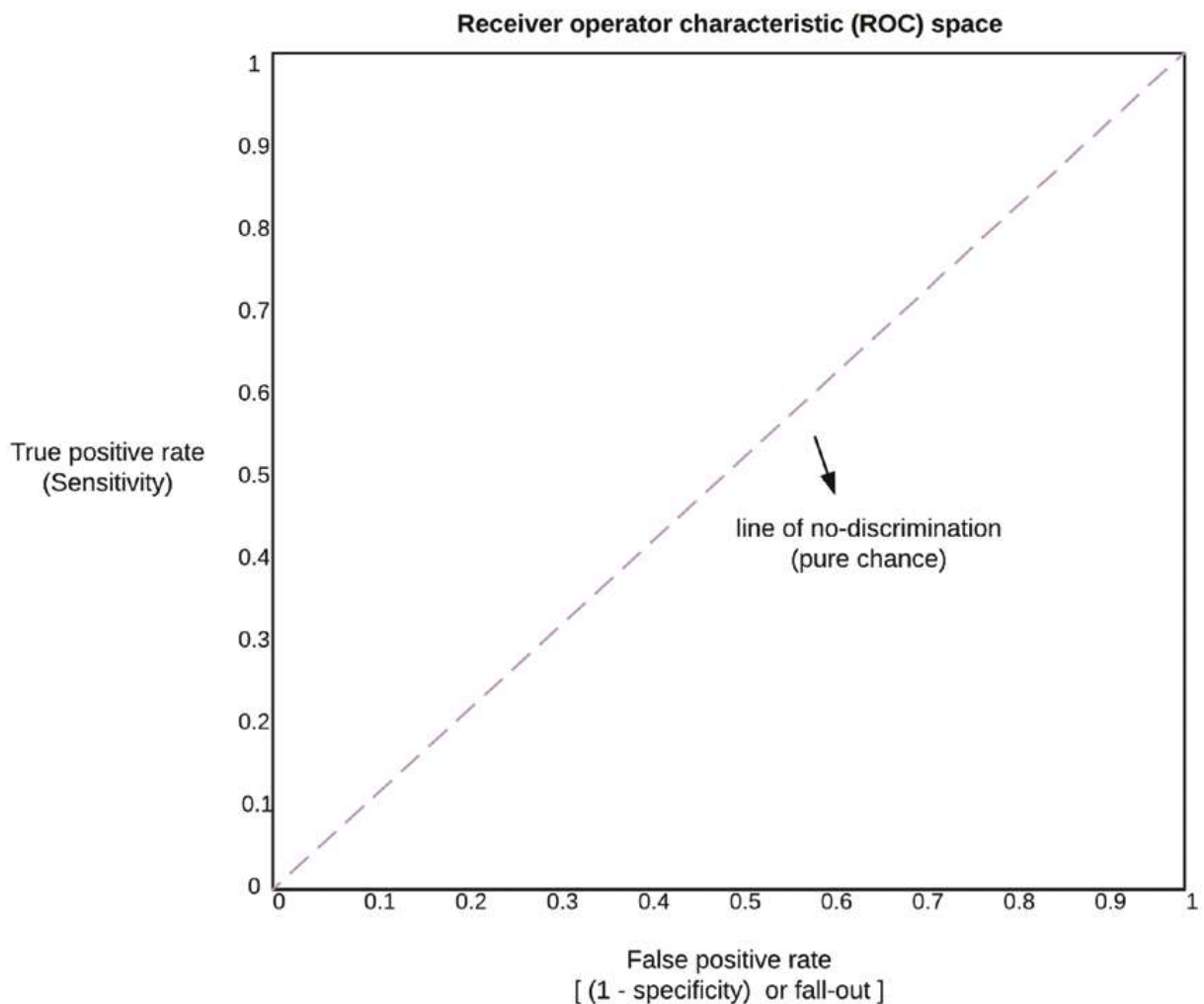


Figure 14-10. Receiver operating characteristic (ROC) space

The AUC-ROC Space

The ROC plot as shown in Figure 14-10 looks like a curve. So, the area under the curve, also known as AUC, is the area underneath the ROC curve. AUC provides a single floating-point number that describes the model's performance, and it is interpreted as follows:

- An AUC value below 0.5 indicates that the model's prediction is worse than a random guess of the targets.
- An AUC value closer to 1 signifies a model that is performing very well by generalizing to new examples on the test dataset.

A ROC curve that is closer to the top-left part of the ROC space (i.e., closer to the value 1) indicates that the model has a good classification accuracy.

The AUC-ROC curve is illustrated in Figure 14-11.

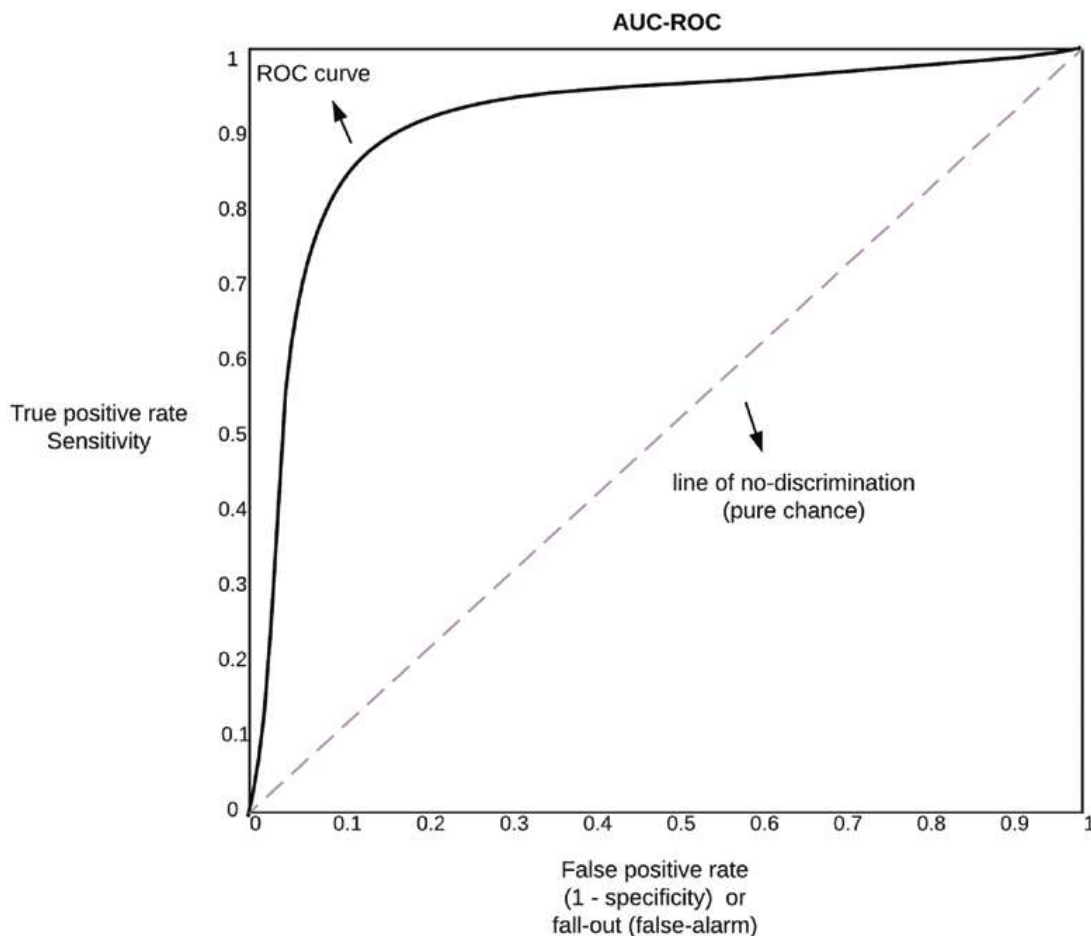


Figure 14-11. AUC-ROC curve

Regression Evaluation Metrics

In this section, we'll go through some of the metrics for evaluating regression machine learning tasks.

Root Mean Squared Error (RMSE)

Root mean squared error also known as RMSE for short is an important evaluation metric in supervised machine learning for regression problems. RMSE computes the error difference between the original value of the target feature and the value predicted by the learning algorithm. RMSE is formally represented by the following formula:

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}}$$

where

- n is the number of samples in the dataset
- y_i is the actual value of the target feature
- $\hat{y}_i$ is the target value predicted by the learning algorithm

Further notes on RMSE:

- Squaring the difference between the actual value and predicted value of the labels $(y_i - \hat{y}_i)^2$ gives the positive deviation (i.e., the magnitude) between the two numbers.
- Dividing by n gives the average of the sum of magnitudes. The square root **returns the results in the same unit of measurement** as the target feature.

An Example of Evaluation with RMSE

Assume we want to predict the price of houses (in thousands of Naira¹), and we have the following dataset (see Figure 14-12).

¹Naira is the currency of Nigeria. It is also symbolized by the code NGN and the sign ₦.

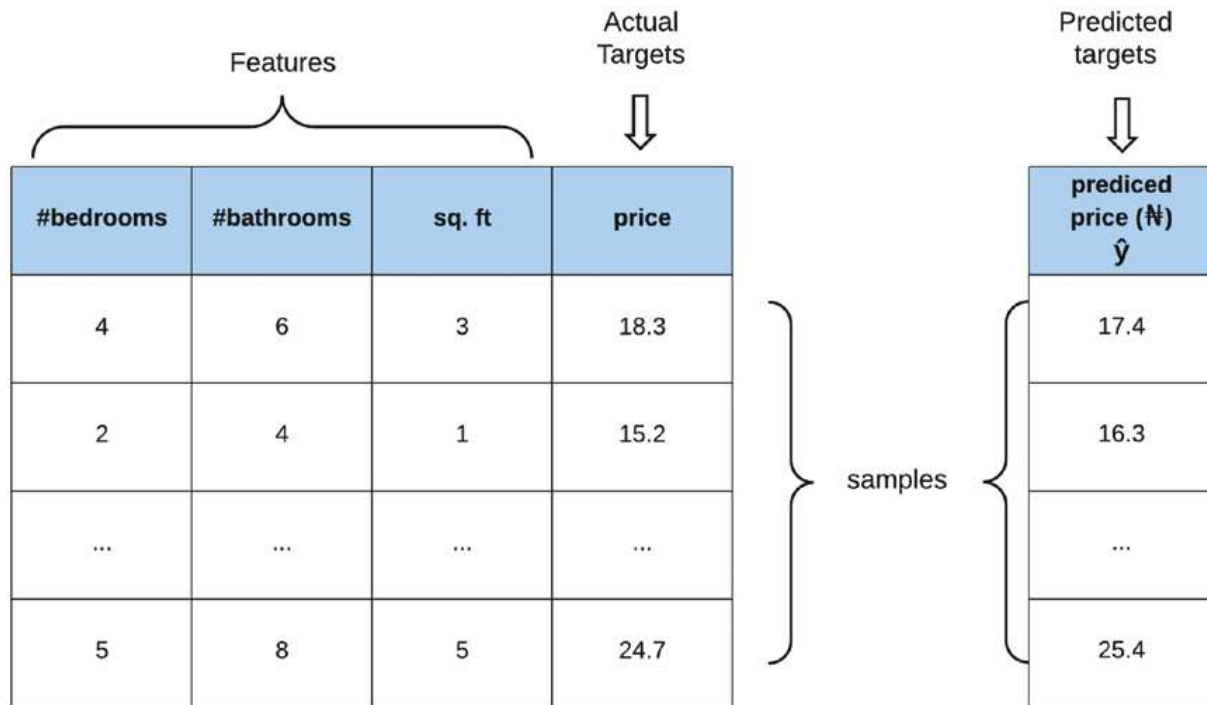


Figure 14-12. RMSE illustration

From the formula given, we calculate the RMSE as follows:

$$\begin{aligned}
 RMSE &= \sqrt{\frac{(18.3 - 17.4)^2 + (15.2 - 16.3)^2 + \dots + (24.7 - 25.4)^2}{3}} \\
 &= \sqrt{\frac{(0.9)^2 + (-1.1)^2 + \dots + (-0.7)^2}{3}} = \sqrt{\frac{0.81 + 1.21 + \dots + 0.49}{3}} \\
 &= \sqrt{\frac{2.51}{3}} = \sqrt{0.8366666666} = \sqrt{MSE} = 0.91469484893
 \end{aligned}$$

The closer the RMSE is to 0, the better the performance of the model. Again, we are most interested in knowing the RMSE on the test data, as this gives us an accurate picture of the performance of our model. In this example, the error difference between the actual price and predicted price of houses made by our learning model is approximately NGN 910 (i.e., $0.91 * 1000$).

Hence, we can calculate the percentage error as

$$\begin{aligned}\%error &= \frac{\text{error difference}}{\text{mean of the actual prices, } \underline{y}} = \frac{0.91}{19.4} \\ &= 0.04690721649 = (0.04690721649 * 100) \approx 4\%\end{aligned}$$

R-squared (R^2)

R-squared, written as R^2 , is another regression error metric that gives us a different perspective into the performance of a learned model. R^2 is also known as the ***coefficient of determination***. The goal of R^2 is to tell us how much of the variance or the variability in the target feature, y , is explained or is captured by the model.

Recall that a model has high variance when it has learned closely the underlying structure of the targets in the dataset. ***Of course, we are mostly concerned with the R^2 metric on test data.*** We typically want the R^2 value on test data to be high. It shows that our model generalizes well to new examples.

Interpretation of R^2

R^2 outputs a value between 0 and 1. Values close to 0 show that variability in the responses are not properly captured by the model, while values close to 1 indicate that the model explains the variability in the observed values. R^2 is calculated using the equation

$$R^2 = 1 - \frac{RSS}{TSS}$$

where

- RSS (i.e., the residual sum of squares) captures the error difference (or the variability) between the actual values and the values predicted by the learning algorithm. The formula is

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- TSS (i.e., the total sum of squares), on the other hand, calculates the variability in the response variable, y . So, for each observation in the dataset, we find the squared difference from the mean of all observation, $\underline{y}$. The formula is

$$TSS = \sum_{i=1}^n (y_i - \underline{y})^2$$

- Hence, $\frac{RSS}{TSS}$ gives us a ratio of how much of the variability in the response variable y **is not explained** by the model.

So, when we say $1 - \frac{RSS}{TSS}$, we reverse the definition to tell us the ratio of the variability in the response variable explained by the model.

An Example of Evaluating the Model Performance with R^2

Using the dataset illustrated in Figure 14-12 and from the formula given earlier, we will calculate R^2 as follows:

$$\begin{aligned} RSS &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \left[(18.3 - 17.4)^2 + (15.2 - 16.3)^2 + \dots + (24.7 - 25.4)^2 \right] \\ &= \left[(0.9)^2 + (-1.1)^2 + \dots + (-0.7)^2 \right] = [0.81 + 1.21 + \dots + 0.49] = 2.51 \end{aligned}$$

while for TSS , we have that the mean of the response variable price, y , is

$$\frac{18.3 + 15.2 + 24.7}{3} = 19.4$$

$$\begin{aligned} TSS &= \sum_{i=1}^n (y_i - \underline{y})^2 = \left[(18.3 - 19.4)^2 + (15.2 - 19.4)^2 + \dots + (24.7 - 19.4)^2 \right] \\ &= \left[(-1.1)^2 + (-4.2)^2 + \dots + (5.3)^2 \right] = [1.21 + 17.64 + \dots + 28.09] = 46.94 \end{aligned}$$

Finally,

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{2.51}{46.94} = 1 - (0.05347251811) = 0.94652748189$$

The result shows that the model does a good job in capturing the variability in the target feature. *Of course, we want to have such good performances on the test dataset.*

Resampling Techniques

This section describes another vital concept for evaluating the performance of supervised learning methods. Resampling methods are a set of techniques that involve selecting a subset of the available dataset, training on that data subset, and then using the remainder of the data to evaluate the trained model.

This process involves creating subsets of the available data into a training set and a validation set. The training set is used to train the model, while the validation set will evaluate the performance of the learned model on unseen data. Typically, this process will be carried out repeatedly to get an approximate measure of the training and test errors.

We will examine three techniques for data resampling and also give some examples of when to use a particular technique. The techniques we'll examine are

- The validation set technique (or train-test split)
- The leave-one-out cross-validation (LOOCV) technique
- The k -fold cross-validation technique

The Validation Set

The validation set is the simplest approach for data resampling, and it involves randomly dividing the dataset into two parts; these are the training set and the validation set. The division can be into two equal sets if you have a big enough dataset, or it could be a 60/40 or 70/30 split.

After splitting, the model is trained on the training set, and its performance is evaluated on the validation set. This process is summarized in the list as follows:

1. Randomly split the dataset into
 - Training set
 - Validation set
2. Train the model on the training set.

3. Evaluate the model performance on the validation set using the appropriate error metric for a regression or classification problem.
4. No. 1 to No. 3 can be repeated.
5. Report the error metric to get the ensemble training and validation set error distribution.

A sample validation set is shown in Figure 14-13.

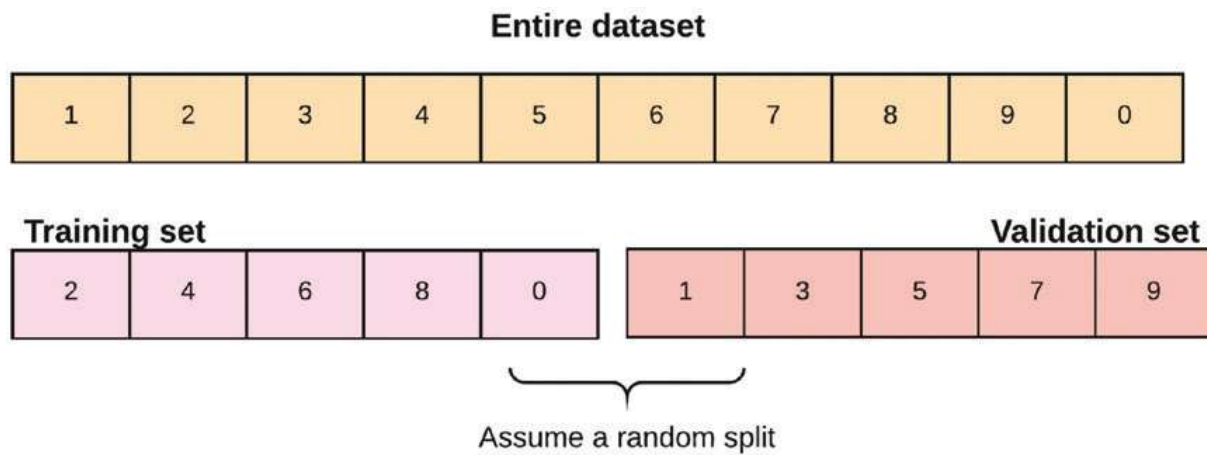


Figure 14-13. *Validation set*

Leave-One-Out Cross-Validation (LOOCV)

The leave-one-out cross-validation approach (commonly referred to as LOOCV) involves dividing the dataset into a training set and a test set. But unlike the validation approach, LOOCV assigns just one example to the test set, and trains the model on the remainder of the dataset. This process is repeated until all the examples in the dataset have been used for evaluating the model.

Assuming we have ten examples in a dataset (let n be used to denote the size of the dataset) to build a learning model. We will train the model using $n - 1$ examples and evaluate the model on just the single remaining example, hence the name leave-one-out. This process is repeated n times for all the examples in the dataset. At the end of the n iterations, we will report the average error estimate.

A sample LOOCV is shown in Figure 14-14.

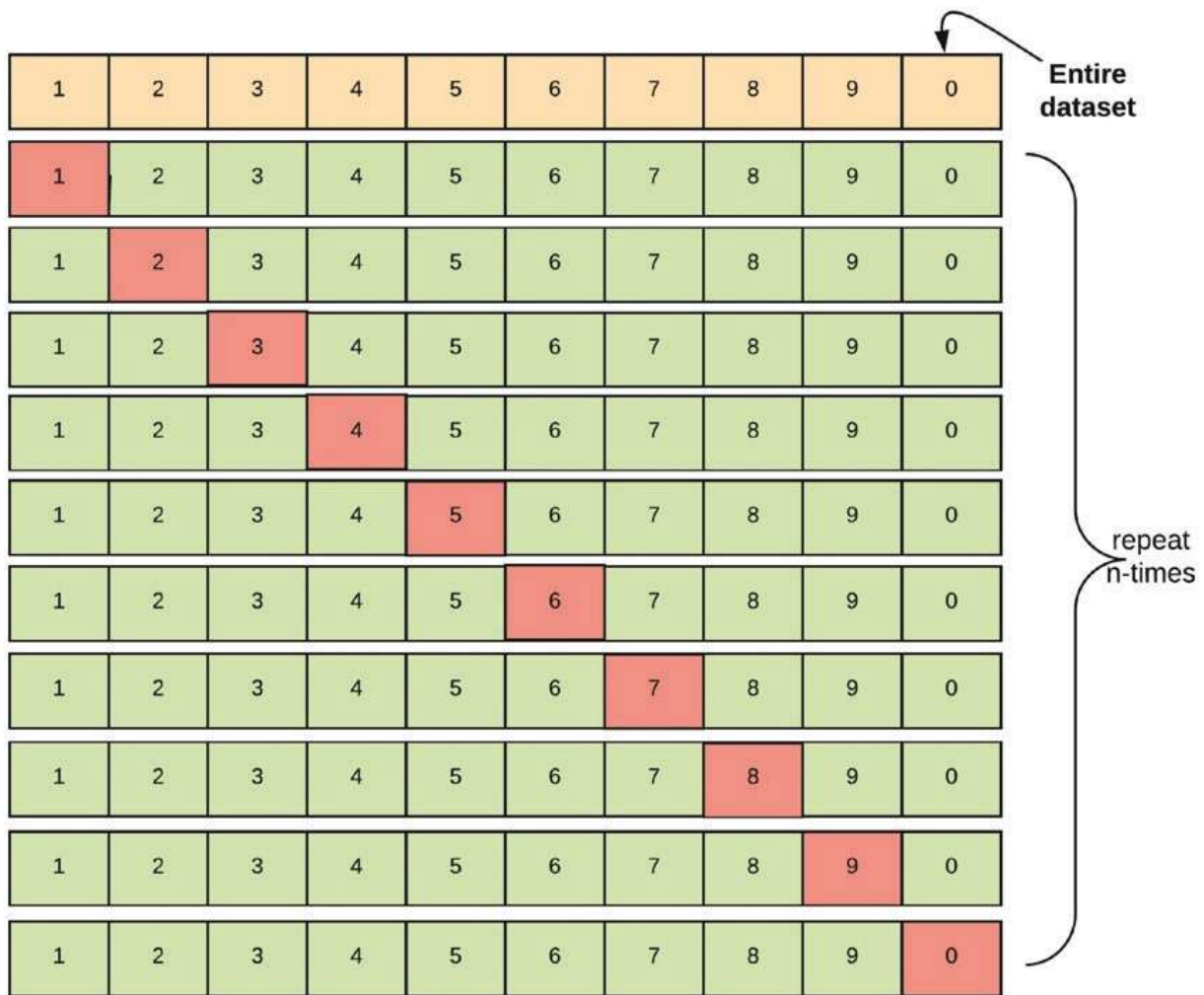


Figure 14-14. LOOCV

The main drawback to using LOOCV is that it is *computationally expensive*. The word *computationally expensive* is when a process takes a lot of computing time and memory to complete its execution.

***k*-Fold Cross-Validation**

k-Fold cross-validation mitigates the computational cost of LOOCV while maintaining its benefits in terms of giving an unbiased estimate of the performance of the learned model when evaluated on validation data.

Let's use the following recipe to explain the idea behind k -fold CV:

- Divide the dataset into k parts or folds. Assume we have a dataset with 20 records; we'll divide the dataset into four parts. See Figure 14-15.

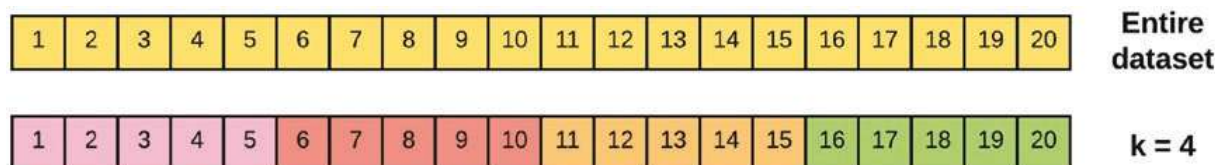


Figure 14-15. Divide your dataset into k parts or folds

- Hold out one of the four splits as a test set, and train the model on the remaining splits. Repeat this until all the splits have been held out for testing. See Figure 14-16.

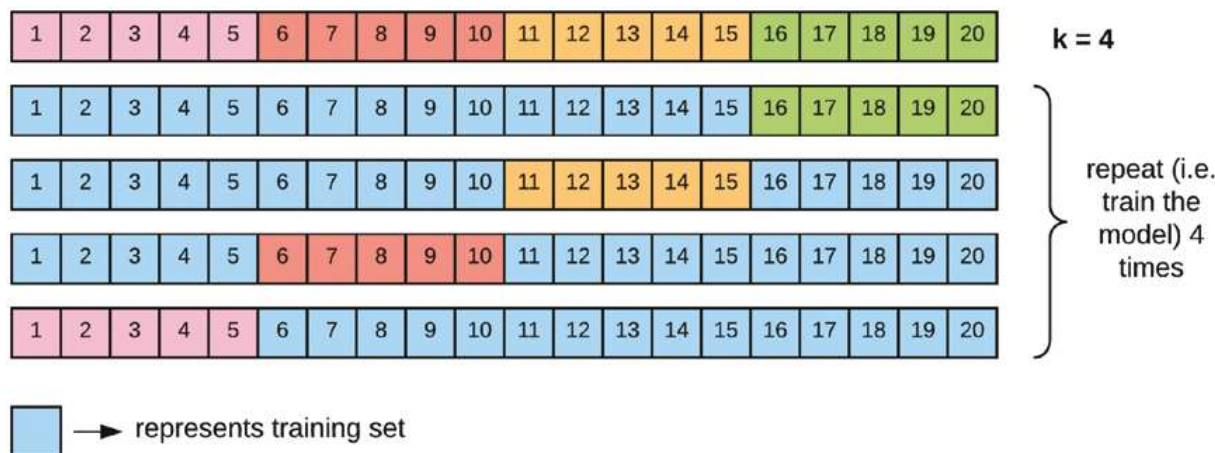


Figure 14-16. Train the model using $k - 1$ example sets or splits

- Report the ensemble error metric.

Note From this explanation, we can see that LOOCV is a special case where $k = n$.

Improving Model Performance

To improve the performance of the model, a few of the techniques to consider are

1. Systematic feature engineering
2. Using ensemble learning methods (we'll discuss more on this in a later chapter)
3. Hyper-parameter tuning of the algorithm

Feature Engineering

In model building, a significant portion of time is spent on feature engineering. Feature engineering is the practice of systematically going through each feature in the dataset and investigating its relevance to the targets.

Through feature engineering, we can cleverly introduce new features by combining one or more existing features, and this can impact the prediction accuracy of the model. Feature engineering can sometimes be the difference between a decent learning model and a competition-winning model.

Ensemble Methods

Ensemble methods combine the output of weaker models to produce a better performing model. Two major classes of ensemble learning algorithms are

- Boosting
- Bagging

In practice, ensemble methods such as Random forests are known to do very well in various machine learning problems and are the algorithms of choice for machine learning competitions.

Hyper-parameter Tuning

When modeling with a learning algorithm, we can adjust certain configurations of the algorithm. These configurations are called hyper-parameters. Hyper-parameters are tuned to get the best settings of the algorithms that will optimize the performance of the model. One strategy is to use a grid search to adjust the hyper-parameters when fine-tuning the model.

Unsupervised Learning

In unsupervised learning, the goal is to build a model that captures the underlying distribution of the dataset. The dataset has no given targets for the input features (see Figure 14-17). So, it is not possible to learn a function that maps a relationship between the input features and the targets as we do in supervised learning.

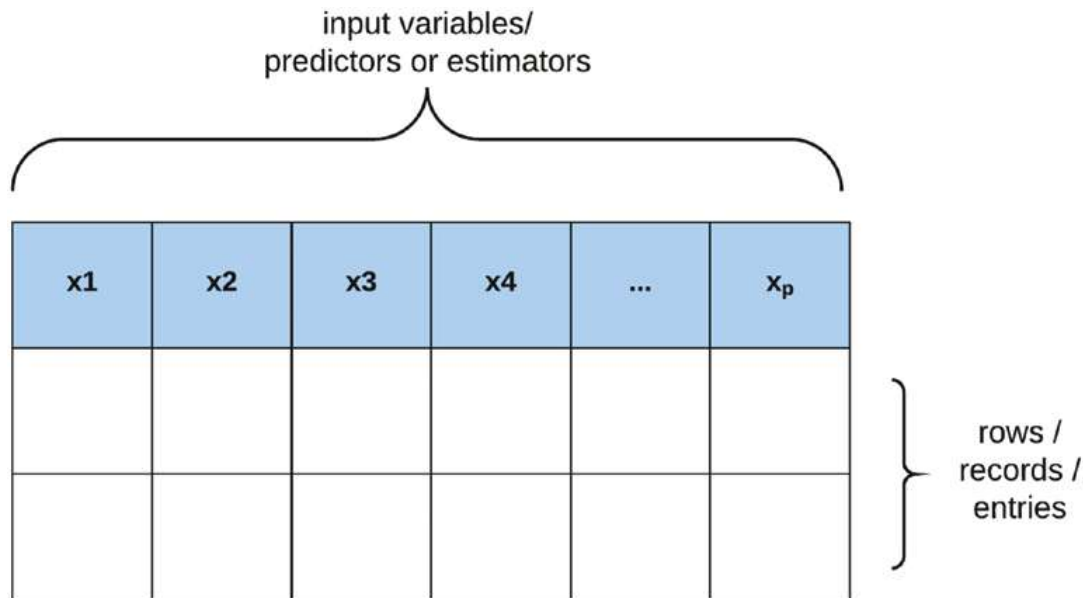


Figure 14-17. *Unsupervised dataset*

Rather, unsupervised learning algorithms attempt to determine the unknown structure of the dataset by grouping similar samples together.

Assume we have a dataset of patients with heart diseases; using unsupervised machine learning algorithms, we can find some hidden sub-groups of patients to help understand more about the disease patterns. This is known as *clustering*.

Also, we can use algorithms like *principal component analysis (PCA)* to compress a large number of features into principal components (that summarizes all the other features) for easy visualization. We will talk more about clustering and principal component analysis in later chapters.

Reinforcement Learning

Reinforcement learning presents an approach to learning that is quite different from what we have seen so far in supervised and unsupervised machine learning techniques. In reinforcement learning, an agent interacts with an environment in a feedback configuration and updates its strategy for choosing an action based on the responses it gets from the environment. An illustration of this scenario is shown in Figure 14-18.

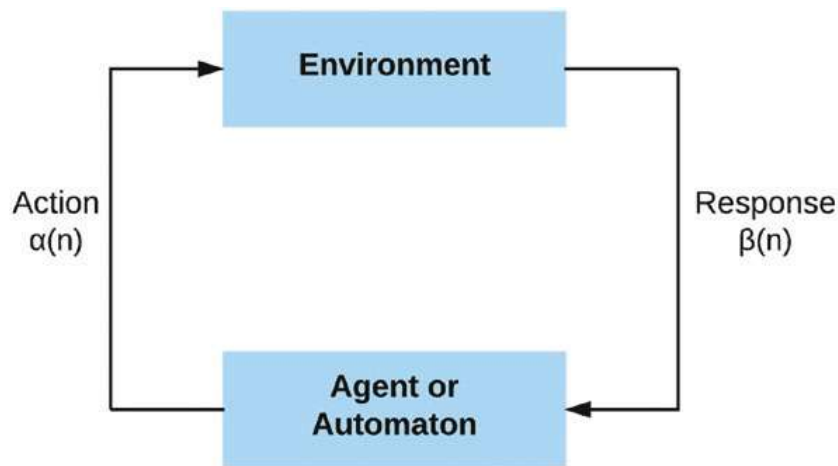


Figure 14-18. Reinforcement learning model

This book will not cover reinforcement learning techniques as it presents a different approach to the problem of learning from random environments that is distinct from the approach used in supervised and unsupervised learning problems.

In this chapter, we covered the three main components of machine learning, which are supervised, unsupervised, and reinforcement learning. The chapter largely focused on the principles for performing supervised machine learning such as framing a problem as a regression or classification task; splitting the dataset into training, test, and validation sets; understanding the bias/variance trade-off and consequently issues of overfitting and underfitting; and the evaluation metrics for assessing the performance of a learning model.

In the next chapter, we will briefly look at the differences between batch and online learning.